Operating Systems - Assignment 1

Evaluate and Justify the Operating-System Design of a Smart Hospital

Tasks 1 to 12 | complete report with code, execution output and analysis | Total 50 marks

Student name	Vimala
Register number	1924260099
Course code	CSA0404 - Operating Systems
Assessment	Assignment 1 (Individual) - K5 Evaluate - 50 marks
GitHub repository	CSA04-1924260099-Assignment1 (add your public link here before submission)
Compiler / OS used	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1) on Ubuntu 24.04.5 LTS, Linux kernel 6.18, 2 vCPUs, Intel Xeon, 8 GB RAM
Compile command	gcc -std=c11 -Wall -Wextra -pthread -O1 -D_GNU_SOURCE file.c -o file

Contents and marks

Task	Topic	Marks	Code and evidence in this report
1	Requirements of each computing environment	4	Ranked tables; measured alarm wake-up latency (latency.c)
2	Type of operating system for each environment	3	Type table; Amdahl and availability calculations
3	Operating-system structures	4	Structure comparison; system call vs message cost (ipc_cost.c)
4	Services an operating system must provide	4	Service matrix; file-protection demonstration (protect_demo.sh)
5	System calls	4	readfile.c with strace output; call path diagram
6	Cost of system calls	4	copy_file.c (TODO completed), strace -c and time tables
7	Process-control system calls	4	shell_job.c (TODO completed), minishell.c, zombie and spawn tests
8	CPU states and protection	3	protection.c and poll_vs_interrupt.c with output
9	Memory hierarchy and I/O	4	chase.c measurements; DMA versus interrupt calculation
10	Measure and compare	4	Tables from strace -c, time, fork versus thread tests
11	Recommended OS plan	4	Final plan table, risks, virtual-machine analysis
12	Day-4 design challenge	8	Impact, evaluation, adaptation, defence

How to read this report

Every task follows the assignment's structure: the discussion points, the **Alternative Concept and Complexity Analysis**, the **Complete the Code** program where one is given, and the **Evidence Expected**. All program output shown is real output from running the code in this repository on Linux. Figures marked *typical* are commonly quoted values for the technology and not my own measurements. Timings come from a small virtual machine, so absolute values vary on other hardware; the ratios and trends are what the arguments rely on.

Repository layout: one folder per task (Task01 ... Task12) holding the C sources, scripts and output files, plus README.md and run_all.sh, which recompiles and re-runs everything.

Scenario assumed by every task

A 500-bed hospital needs bedside monitors (alarm within 10 ms), nurse tablets, doctor workstations, a never-stopping patient database, a multiprocessor imaging server for MRI and a link to two partner hospitals. Growth path: 1 ward (40 monitors, 10 tablets) to 1 hospital (500 monitors, 120 tablets, 80 workstations) to a group of 3 hospitals. The IT proposal of one general-purpose OS everywhere is evaluated and challenged throughout.

TASK 1 - Evaluate the Requirements of Each Computing Environment

4 marks | K5 Evaluate

1.1 Requirements, ranked with justification

Rank 1 is the requirement that must never be sacrificed in that environment. The five requirements are response time (R), reliability (L), throughput (T), battery (B) and security (S).

Environment	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5	Justification of the top two
Bedside monitor	R: alarm in 10 ms	L	S	B	T	A late alarm directly harms the patient; a monitor that fails silently is as bad as a slow one. Data volume is tiny (a few readings per second), so throughput matters least.
Nurse tablet	B: lasts a shift	S	R	L	T	Staff cannot work with a dead tablet; tablets are lost or stolen easily, so patient data on them must be protected.
Doctor workstation	S	L	R	T	B	Shared desks, many staff and sensitive records make access control first; they sit on mains power, so battery is irrelevant.
Database server	L: never stops	S	T	R	B	Every ward depends on it; it holds the most sensitive data. Look-ups need to be quick but a second's delay is tolerable.
Imaging server	T: MRI data	L	S	R	B	A 500 MB scan must be processed quickly by many CPUs; a crash loses processing work but not patient safety.
Partner link	S	L	T	R	B	Records cross an organisation boundary, so authentication and encryption come first; the link must stay up for shared care.

Concrete targets used in later tasks

Environment	Response time	Reliability	Throughput	Battery	Security need
Monitor	Hard deadline 10 ms	Continuous, watchdog restart in under 1 s	About 1 KB/s per monitor	Mains plus 4 h backup	Authenticated, signed firmware
Tablet	UI under 100 ms	Restart tolerated	Low	10-12 h, one full shift	Encrypted storage, remote wipe
Workstation	UI under 100 ms	99.9%	Moderate	Mains	Login, access control, audit
Database	Look-up under 50 ms	99.99% or better	Thousands of transactions per minute	Mains plus UPS	Encryption, ACLs, audit
Imaging	Minutes per scan	99.9%	Hundreds of MB per scan	Mains	Restricted access
Partner link	Seconds	99.9%	Moderate	Mains	Mutual authentication, encryption

1.2 Conflicting requirements

- **Fast response versus many users:** time-sharing gives every user a slice, so a monitor sharing a CPU with a workstation's programs may wait. Separate environments, or a real-time scheduler with priorities, remove the conflict.
- **Battery versus response time:** keeping the CPU awake for instant response drains the tablet; sleeping saves power but adds wake-up delay. Tablets accept 100 ms; monitors cannot, so they use mains with backup.
- **Security versus response time:** encryption and access checks cost CPU time. The monitor's alarm path is therefore kept free of heavy checks, and security is applied at the network edge.
- **Throughput versus reliability:** the database waits for data to reach disk before committing, which lowers throughput but prevents loss.
- **Throughput versus fairness:** the imaging server wants every CPU for one scan, while the database wants to serve many small requests; they are placed on separate machines.

1.3 Requirements critical to patient safety

Critical: (1) the monitor's 10 ms alarm and its continuous operation; (2) database availability and correctness (wrong or missing allergy data is a safety issue); (3) integrity of records on the partner link. Important but not immediately life-critical: workstation login security, tablet battery, imaging throughput. This is why the monitors and the database receive the strictest guarantees in the plan.

1.4 Lessons from operating-system history

Historical stage	Lesson	Where it applies in the hospital
Batch systems	Run jobs one after another with no interaction to keep the CPU busy; throughput is the goal.	Overnight backup of the database and queued MRI processing jobs.
Time-sharing	Give each user a short time slice so many people feel they have the whole machine; needs protection between users.	Database server and imaging server with many users; the same ideas make a monitor unsuitable if it must share slices with others.
Personal computers	One user, a friendly interface, responsiveness over utilisation.	Doctors' desktop workstations.
Mobile / handheld	Power is the scarce resource; touch interface; apps are sandboxed.	Nurses' tablets.
Real-time and embedded	Correctness depends on meeting deadlines; small, predictable kernels.	Bedside monitors, and later the home devices.
Distributed and clustered	Many machines cooperate; failure of one must not stop the service.	Database cluster and the partner-hospital link.

1.5 Alternative Concept and Complexity Analysis: one general-purpose OS everywhere

Solution if the whole hospital used one general-purpose (GP) OS. Every monitor would run a full time-sharing kernel such as a desktop Linux. The alarm program becomes an ordinary process competing with logging, networking and update tasks for the CPU. I measured the real wake-up lateness of a 1 ms periodic task (program `latency.c`): how late the task actually runs after the time it asked for.

Source: `Task01/latency.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sched.h>
#include <string.h>
static int cmp(const void*a,const void*b){double x=*(double*)a,y=*(double*)b;return (x>y)-(x<y);}
/* Alarm-wake-up latency: ask to wake every 1 ms, measure how late we actually wake (us). usage: latency [fifo] */
int main(int argc, char **argv)
{
    if (argc > 1 && !strcmp(argv[1], "fifo")) { struct sched_param sp = {.sched_priority = 80};
        if (sched_setscheduler(0, SCHED_FIFO, &sp) != 0) { perror("SCHED_FIFO"); return 1; } }
    int N = 5000; double *l = malloc(N*sizeof(double)); struct timespec t, now; clock_gettime(CLOCK_MONOTONIC, &t);
    for (int i = 0; i < N; i++) {
        t.tv_nsec += 1000000; if (t.tv_nsec >= 1000000000) { t.tv_nsec -= 1000000000; t.tv_sec++; }
        clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &t, NULL); clock_gettime(CLOCK_MONOTONIC, &now);
        l[i] = (now.tv_sec - t.tv_sec)*1e6 + (now.tv_nsec - t.tv_nsec)/1e3;
    }
    double s = 0, m = 0; for (int i = 0; i < N; i++) { s += l[i]; if (l[i] > m) m = l[i]; }
    qsort(l, N, sizeof(double), cmp);
    printf("wake-up lateness over %d x 1 ms: avg %.1f us, p99 %.1f us, max %.1f us\n", N, s/N, l[(int)(N*0.99)], m);
    return 0;
}
```

Condition (5000 wake-ups of a 1 ms timer)	Average	99th percentile	Worst case
GP scheduler, idle machine	80.8 us	379.2 us	1047.5 us
GP scheduler, 4 CPU-hungry programs on 2 CPUs	137.3 us	3089.7 us	8040.4 us
Real-time priority (SCHED_FIFO), same load	13.4 us	39.1 us	2108.5 us
Real-time priority, idle machine	20.9 us	68.6 us	323.5 us

Output: general-purpose scheduler under load (file `Task01/output_loaded.txt`)

```
wake-up lateness over 5000 x 1 ms: avg 137.3 us, p99 3089.7 us, max 8040.4 us
```

Evaluation. With ordinary scheduling, load pushed the worst delay from about 1 ms to about 8 ms and the 99th percentile to about 3 ms; the delay is **unbounded in principle** because it grows with the number of competing programs. Eight milliseconds already consumes 80% of the 10 ms budget before the alarm is even evaluated,

and on a busier machine it could exceed 10 ms. A real-time priority removes almost all of this, but even then my virtual machine showed occasional spikes of about 2 ms because of the hypervisor and other kernel activity, which a true RTOS on bare hardware does not have. A hard real-time OS gives a *guaranteed* worst case (set by the design, typically tens of microseconds).

Criterion	One GP OS on every device	Different OS per environment
Worst-case alarm time	Unbounded; measured about 8 ms under modest load and rising with load, O(load)	Bounded and constant, O(1), by fixed-priority scheduling
Memory per monitor (typical)	About 512 MB to 1 GB RAM and several GB storage for a full Linux	FreeRTOS kernel about 6-12 KB flash and a few KB RAM; QNX microkernel under 1 MB
Total memory for 500 monitors	500 x 1 GB = about 500 GB	500 x about 1 MB = about 0.5 GB
Administration effort	One image, one patch process, one skill set (advantage); but one bug or patch affects every device at once	Several images and skills (cost); faults and patches are contained per environment
Battery on tablets	Desktop services keep the CPU busy and drain the battery	Handheld OS suspends unused services

Judgement: the single-OS proposal is cheaper to administer but fails the clinical staff's first concern (alarm delay) and their third (tablet battery). Different OS types per environment are justified; administration cost is reduced by choosing Linux-based systems wherever possible.

TASK 2 - Judge the Type of Operating System for Each Environment

3 marks | K5 Evaluate

2.1 Comparison of operating-system types

Type	Key characteristics	Hospital use
Mainframe	Batch and transaction processing, very high I/O capacity, expensive, very reliable	Not chosen: cost is too high for 500 beds; the same reliability is obtained from a small cluster
Desktop	Single user, rich interface, time-sharing for a few programs	Doctor workstations
Multiprocessor (SMP / AMP)	Several CPUs share memory and one OS; higher throughput	Imaging server
Distributed	Separate computers cooperate over a network, resources shared	Link with the partner hospitals; shared records
Clustered	Several nodes act as one; fail-over and load sharing	Database server
Hard real-time	A missed deadline is a failure; bounded latency	Bedside monitors
Soft real-time	Deadlines are goals; occasional miss is tolerable	Voice or video calls, nurse-call notifications
Handheld	Small memory, battery aware, touch, sandboxed apps	Nurses' tablets

2.2 Best-fit type with rejected alternatives

Environment	Chosen type	Rejected alternatives and reasons
Bedside monitors	**Hard real-time OS** (FreeRTOS, or QNX where certification is needed)	Desktop or handheld OS: scheduling delay is unbounded (Task 1 measurements); soft real-time: a missed alarm is unacceptable.
Nurse tablets	**Handheld OS** (Android)	Desktop OS: too much memory and battery use (Task 4 alternative); RTOS: no rich touch interface or app support.
Doctor workstations	**Desktop OS** (Linux or Windows)	Handheld: poor for long records and multi-window work; mainframe terminals: costly and inflexible.
Database server	**Clustered OS** with fail-over	Single large machine: single point of failure (see 2.5); mainframe: cost.
Imaging server	**Symmetric multiprocessing (SMP)** Linux	Asymmetric multiprocessing: master CPU becomes a bottleneck; a single CPU: too slow for MRI.
Partner link	**Distributed system** over secured network	A shared physical machine: not possible across organisations; no security boundary.

2.3 Symmetric versus asymmetric multiprocessing for the imaging server

In **SMP** every CPU runs the same kernel and any CPU can run any task or handle any interrupt, so MRI slices can be spread evenly over all cores and one core failing does not stop the system. In **AMP** a master CPU runs the OS and assigns work to slaves; it is simpler but the master becomes a bottleneck and a single point of failure. MRI reconstruction splits naturally into many independent slices, so SMP is the better fit. The cost of SMP is kernel lock contention, which is why the speed-up is sub-linear (below).

2.4 Clustered database with fail-over

Two nodes share storage or replicate data; a heartbeat message tells the standby that the active node is alive. If the heartbeat stops for a few seconds the standby takes over the service address and recovers the journal. Updates are done by patching the standby first, failing over, then patching the former active node, so the database does not stop during updates (the clinical staff's second concern).

2.5 Alternative Concept and Complexity Analysis: one large multiprocessor instead of a cluster

Solution. Buy one machine with many CPUs and much memory and run the database on it. **Availability:** if one machine is 99.9% available, downtime is 0.001 x 8760 h = **8.76 hours a year**, and any hardware fault stops everything (single point of failure). A two-node cluster of cheaper machines each 99.5% available fails only when both are down: $1 - (0.005)^2 = 99.9975\%$, i.e. **about 13 minutes a year**, with fail-over in seconds.

Throughput growth with processors (time). By Amdahl's law with a parallel fraction f = 0.95 the speed-up on p processors is S(p) = 1 / ((1 - f) + f / p):

Processors p	1	2	4	8	16	32	limit
Speed-up S(p)	1.00	1.90	3.48	5.93	9.14	12.55	20.0

Doubling the processors from 16 to 32 gives only 37% more throughput, because the 5% serial part and lock contention dominate: time T(p) = T(1) x [(1 - f) + f / p]. In the cluster, active-passive adds no throughput but gives availability; active-active partitioning (sharding) can scale throughput almost linearly because nodes share nothing.

Criterion	Single large multiprocessor	Cluster with fail-over
Availability	About 99.9%, single point of failure, 8.76 h/yr down	99.9975% (13 min/yr), recovers in seconds
Cost	Price grows faster than capacity for large SMP machines	Commodity nodes; cost grows about linearly, but needs 2x hardware
Throughput (time)	Sub-linear in processors (Amdahl, lock contention)	Near-linear if data is partitioned; active-passive adds none
Memory (space)	One large shared memory, simple, but a hard upper limit	Memory per node; data may be duplicated, about 2x space for replicas

Judgement: the cluster is chosen for the database (availability dominates); SMP is chosen for the imaging server (throughput dominates and a short outage is tolerable).

TASK 3 - Evaluate Operating-System Structures

4 marks | K5 Evaluate

3.1 Structures compared against the hospital's requirements

Structure	Reliability	Performance	Ease of updating	Examples
Simple / monolithic	Low: a fault in any driver can crash the whole kernel	Highest: calls are plain function calls inside one address space	Hard: rebuild or reboot the whole kernel	MS-DOS, classic UNIX
Layered	Medium: each layer uses only the one below, easy to verify	Lower: a request passes through several layers	Medium: change a layer without touching the others	THE system, network protocol stacks
Microkernel	High: drivers and file servers run in user mode; a failed server can be restarted	Lower: messages between servers cost more than calls (measured below)	High: servers can be replaced while running	QNX, Minix, L4
Modular (loadable modules)	Medium: modules run in kernel mode, so a bad module can still crash	High, close to monolithic	High: modules load and unload without reboot	Linux, Solaris
Hybrid	Medium to high	High	High	Windows NT, macOS, Android on Linux

3.2 How real systems are structured

System	Structure
Linux	Monolithic kernel with loadable kernel modules: scheduler, memory, file systems and most drivers run in kernel mode, but modules can be added or removed at run time.
Windows	Hybrid: a microkernel-style executive with layered components, many services (graphics, file systems) also run in kernel mode for speed.
Android	Layers: applications, Java/Kotlin framework and runtime, native libraries, hardware abstraction layer, Linux kernel.
FreeRTOS	Very small kernel library (scheduler, queues, semaphores, timers) linked into one program with the application; no user/kernel split on many boards.
QNX	True microkernel (scheduling, IPC, interrupts); file systems, drivers and networking are separate user-mode processes that can be restarted.

3.3 Defended choices for two environments

Patient monitor: microkernel / small real-time kernel. The monitor runs few functions, so the extra message cost does not matter against a 10 ms budget, while isolation matters greatly: a faulty display or network driver must not stop the alarm task. Small code size also makes the kernel easier to test and certify.

Database server: monolithic Linux with loadable modules. It needs the highest I/O and file-system speed, mature drivers and journaling file systems, and live patching of modules. Reliability is obtained at the cluster level (fail-over) instead of inside the kernel.

3.4 Alternative Concept and Complexity Analysis: monolithic kernel on the monitors

Solution. Put sensor driver, alarm logic, display and network stack together in one monolithic kernel. A service request is then a single system call. In a microkernel the same request is a message to a server process and a reply. I measured both on Linux (program `ipc_cost.c`): a system call versus a request and reply between two processes through pipes, which models message passing between servers.

Source: `Task03/ipc_cost.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <time.h>
#include <sys/syscall.h>
#include <sys/wait.h>
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return t.tv_sec*1e9+t.tv_nsec;}
/* Monolithic request = 1 system call.  Microkernel request = message to a server process and reply (2 context switches). */
int main(void)
{
```

```
    int N = 200000; double t = now();
    for (int i = 0; i < N; i++) syscall(SYS_getppid);
    double sc = (now()-t)/N;
    int a[2], b[2]; pipe(a); pipe(b); char c = 'x';
    if (fork() == 0) { for (int i = 0; i < N; i++) { read(a[0], &c, 1); write(b[1], &c, 1); } _exit(0); }
    t = now();
    for (int i = 0; i < N; i++) { write(a[1], &c, 1); read(b[0], &c, 1); }
    double ipc = (now()-t)/N; wait(NULL);
    printf("monolithic service request (1 system call)      : %8.0f ns\n", sc);
    printf("microkernel-style request (message to server+reply): %8.0f ns  (%.0fx)\n", ipc, ipc/sc);
    return 0;
}
```

Output (file `Task03/output.txt`)

```
monolithic service request (1 system call)      :       83 ns
microkernel-style request (message to server+reply):    19747 ns  (237x)
```

Criterion	Monolithic kernel	Microkernel
Time per service request	One system call: about 83 ns measured, cost O(1), c_1	Message to server and reply: about 19.7 us measured here (237x). Real microkernels such as QNX or L4 optimise IPC to roughly 1 us or less, so the realistic penalty is about 2-10x. Cost O(1) with larger constant c_2
Effect on the 10 ms alarm	Negligible (0.001% of budget)	Even 20 us is only 0.2% of 10 ms, so it is acceptable
Kernel memory (typical)	Linux kernel about 10-30 MB plus modules	Microkernel about 10-300 KB; servers add to this in user mode; FreeRTOS about 10 KB
Faulty driver	A bug in a kernel-mode driver can overwrite kernel data and **crash the whole system** (kernel panic)	The driver is a user-mode process; the fault is contained and the server is restarted in milliseconds, the alarm task keeps running

Total request time for k requests per alarm cycle is k x c, so even with k = 10 requests and c_2 = 20 us the overhead is 0.2 ms, well inside the 10 ms deadline. Space complexity: monolithic kernel size grows with every driver included, O(number of drivers) in privileged memory; a microkernel keeps privileged code constant and puts the growth in unprivileged servers.

Judgement: the performance advantage of a monolithic kernel is real (about 240x on this test) but irrelevant for the monitor, whereas the fault-containment advantage of the microkernel is directly relevant to patient safety. The microkernel / small RTOS is retained for the monitors.

TASK 4 - Evaluate the Services an Operating System Must Provide

4 marks | K5 Evaluate

4.1 Service table for each environment

Full = complete service, Min = minimal version, No = removed. The assignment's services are user interface, program execution, I/O, file system, communication, error detection, resource allocation, accounting, protection and security.

Service	Monitor	Tablet	Workstation	Database	Imaging	Partner link
User interface	Min (small display, buttons)	Full touch	Full GUI	No (remote shell only)	No	No
Program execution	Full (fixed tasks)	Full	Full	Full	Full	Full
I/O operations	Full (sensors, alarm)	Full	Full	Full (disks)	Full (DMA)	Network I/O
File system	No (flash log only)	Min	Full	Full, journaling	Full, large files	Min
Communication	Min (one secured channel)	Full (Wi-Fi)	Full	Full	Full	Full
Error detection	Full (watchdog, sensor checks)	Full	Full	Full (ECC, journal)	Full	Full
Resource allocation	Min (fixed priorities)	Full	Full	Full	Full (CPU, memory)	Min
Accounting	No	Min	Min	Full (audit)	Min	Full (audit)
Protection and security	Min (signed firmware)	Full (encryption, wipe)	Full (login)	Full (ACLs, encryption)	Full	Full (authentication, TLS)

4.2 Services removed from the bedside monitor

Removing the general file system, multi-user login, accounting, the graphical desktop, dynamic module loading, printing and most of the network stack makes the monitor image smaller and faster: less code to load and fewer interrupts and background tasks that could delay the alarm, and a smaller attack surface (less code means fewer bugs to exploit). What must remain: scheduling with priorities, interrupt handling, timers, a driver for the sensor and alarm, a secured communication channel, a watchdog and error detection.

4.3 Services essential for the database server

A journaling file system (survives power loss), process and memory management for many concurrent transactions, fast I/O with caching, communication for clients and the cluster heartbeat, error detection (ECC memory, disk checks, journal recovery), accounting and audit logs (who read which record), and strong protection (user IDs, ACLs, encryption).

4.4 How the OS protects one patient's records from another user's programs

Layers of protection: (1) **address-space isolation**, so a program cannot read another program's memory (Task 8); (2) **user IDs and file permissions or ACLs**, checked by the kernel on every open(); (3) **authentication** at login; (4) **audit logs**; (5) optional mandatory access control (SELinux). Evidence: the script below creates a record readable only by its owner and shows another user's program being refused.

Source: `Task04/protect_demo.sh`

```
#!/bin/bash
# Evidence for Task 4: the OS file protection stops another user's program reading a patient record
echo "Patient 101: diagnosis confidential" > /tmp/patient101.rec
chown root:root /tmp/patient101.rec; chmod 600 /tmp/patient101.rec
ls -l /tmp/patient101.rec
echo "--- owner (root) reads:";  cat /tmp/patient101.rec
echo "--- another user (nobody) tries to read:"; su -s /bin/sh nobody -c 'cat /tmp/patient101.rec'; echo "exit status $?"
chmod 640 /tmp/patient101.rec; chgrp nogroup /tmp/patient101.rec
echo "--- after giving group 'nogroup' read permission (like a ward group):"; su -s /bin/sh nobody -c 'cat /tmp/patient101.rec'
rm -f /tmp/patient101.rec
```

Output (file `Task04/output.txt`)

```
-rw------- 1 root root 36 Nov  5 05:25 /tmp/patient101.rec
--- owner (root) reads:
Patient 101: diagnosis confidential
--- another user (nobody) tries to read:
cat: /tmp/patient101.rec: Permission denied
exit status 1
--- after giving group 'nogroup' read permission (like a ward group):
Patient 101: diagnosis confidential
```

The kernel refused user *nobody* with **Permission denied** until the group permission was deliberately granted, which also shows how a ward group could be given access to its own patients only.

4.5 Alternative Concept and Complexity Analysis: tablets with a full desktop OS

Solution. Install a full desktop OS with every service enabled (printing, multi-user accounts, full GUI desktop, indexing, background updates) on each tablet. The figures below are *typical* values for the two classes of system, not my measurements.

Criterion	Reduced handheld OS (typical)	Full desktop OS, all services (typical)	Effect
Storage (space)	About 4-8 GB for system	About 20-40 GB	Roughly 4-6x more flash used, and 120 tablets x 25 GB extra = 3 TB wasted
Memory at idle (space)	About 0.5-1 GB	About 2-4 GB	Little memory left for the clinical app on a 4 GB tablet
Start-up time	About 15-30 s	About 45-90 s	Slower to begin a ward round or recover from a crash
Battery life (time)	About 10-14 h	About 4-6 h	Does not last a 12-hour shift (the clinical staff's third concern)

Power is roughly proportional to the work done: background services keep the CPU out of deep sleep, so energy over a shift $E = P_{avg} \times t$ grows with the number of active services, O(number of services). Boot time grows with the number of services started, O(services). The reduced OS therefore wins on both time and space, and also reduces the attack surface.

Judgement: a reduced handheld OS is justified for tablets; removing services is both a performance and a security decision.

TASK 5 - Examine System Calls

4 marks | K5 Evaluate

5.1 Categories of system calls

Category	Purpose	Linux examples	Hospital example
Process control	Create, end, wait for processes; load programs	fork, execve, wait4, exit, clone	Shell starts a backup job; monitor starts its tasks
File management	Create, open, read, write, close files	open, read, write, close, lseek, unlink	Database reads and writes records
Device management	Request, read, write, release devices	ioctl, read, write on device files, mmap	Monitor reads the sensor; MRI scanner transfer
Information maintenance	Get or set time, process and system data	getpid, gettimeofday, uname, sysinfo	Time-stamping a reading
Communication	Message passing or shared memory between processes or hosts	pipe, socket, send, recv, shmget	Monitor sends reading to nurse station
Protection	Control access to resources	chmod, chown, setuid, umask	Restricting a patient record to a ward group

5.2 What happens step by step when a program calls read()

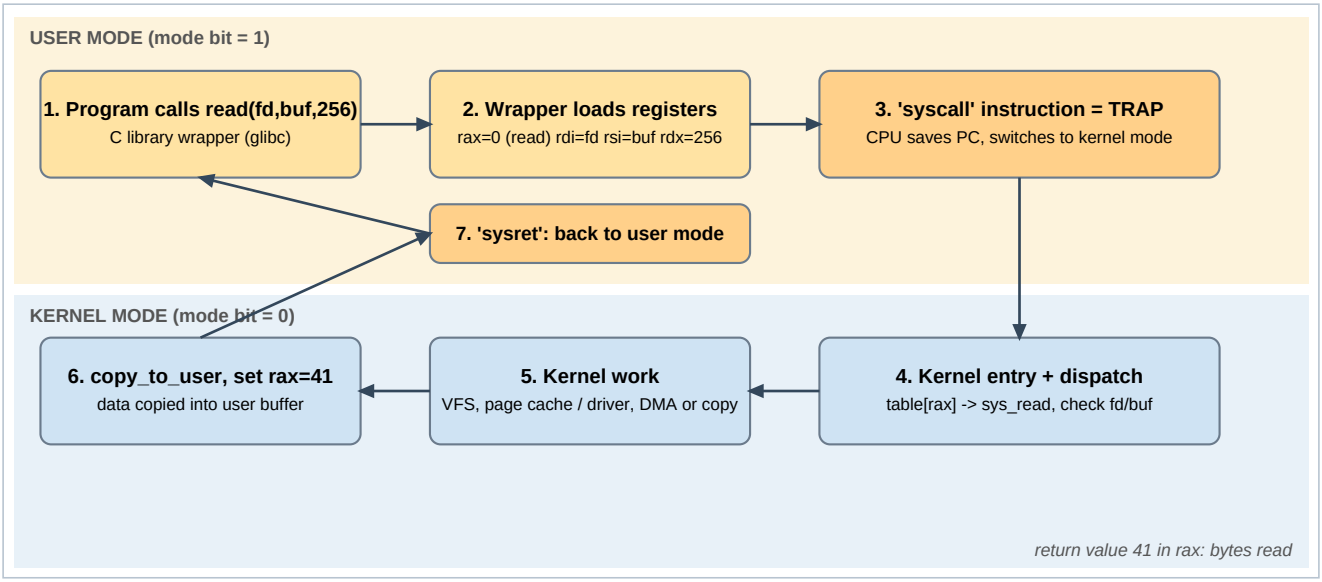


Figure 5.1 - Path of a read() call. The shaded zones show the mode change at the trap and the return.

1. **Library call:** the program calls the C-library wrapper read(fd, buf, count). The wrapper is an ordinary function in user mode.
2. **Parameters set up:** the wrapper puts the system-call number (0 for read on x86-64) and the arguments into registers.
3. **Trap:** the wrapper executes the special instruction (`syscall`). The CPU saves the return address and flags, and switches to **kernel mode** (mode bit 0), jumping to a fixed kernel entry point. A user program cannot jump anywhere else in the kernel.
4. **Mode switch and dispatch:** the kernel saves user registers, looks up the call number in the system-call table and calls sys_read, after validating the file descriptor and checking that buf is a legal user address.
5. **Kernel work:** the virtual file system finds the file, the data come from the page cache or from the disk driver (using DMA and an interrupt when the disk is done); the process may sleep meanwhile and another process runs.
6. **Return:** the kernel copies the data into the user buffer (copy_to_user), places the count in the return register and executes `sysret`, returning to user mode. The wrapper returns the count, or -1 and sets errno.

5.3 How parameters are passed to the kernel

Method	How it works	Used when
Registers	Call number and up to six arguments in CPU registers. Linux x86-64: rax = number, rdi, rsi, rdx, r10, r8, r9 = arguments	The normal Linux method: fastest, no memory access needed
Block (table) in memory	Arguments placed in a structure in memory; the address of the block is passed in a register	More than six arguments or large data, e.g. clone3 and openat2 pass a pointer to a struct; Solaris and some older systems
Stack	The program pushes parameters on the stack; the kernel pops them	Some other systems (for example classic BSD conventions); no limit on count but slower

5.4 strace evidence from a real program

The program below reads a file with read() and writes it with write(). `strace` intercepts every system call the process makes and prints its name, arguments and return value.

Source: `Task05/readfile.c`

```
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <unistd.h>
/* Reads a file with read() and writes it to stdout with write(): shows the system calls in strace */
int main(int argc, char **argv)
{
    char buf[256];
    int fd = open(argc > 1 ? argv[1] : "patient.txt", O_RDONLY);
    if (fd < 0) return 1;
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0)
        write(STDOUT_FILENO, buf, (size_t)n);
    close(fd);
    return 0;
}
```

Program output (file `Task05/output_run.txt`)

```
Patient 101: heart rate 72 bpm, SpO2 98%
```

strace output (the lines belonging to the program's own work) (file `Task05/output_strace.txt`)

```
close(3)                                = 0
openat(AT_FDCWD, "patient.txt", O_RDONLY) = 3
read(3, "Patient 101: heart rate 72 bpm, "..., 256) = 41
write(1, "Patient 101: heart rate 72 bpm, "..., 41) = 41
read(3, "", 256)                        = 0
close(3)                                = 0
exit_group(0)                           = ?
```

Reading the trace: `openat` returns file descriptor 3; `read(3, ..., 256) = 41` asks for up to 256 bytes and the kernel returns the file's 41 bytes; `write(1, ..., 41) = 41` sends them to standard output (descriptor 1); the second read returns 0, the end-of-file signal; `close` and `exit_group` end the program. Before main() the dynamic loader makes about 30 more calls (mmap, mprotect, fstat) to load libc, which is why a three-line program shows 36 trace lines.

strace -c summary (first lines) (file `Task05/output_strace_c.txt`)

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         3           read
  0.00    0.000000           0         1           write
  0.00    0.000000           0         3           close
  0.00    0.000000           0         2           fstat
  0.00    0.000000           0         8           mmap
  0.00    0.000000           0         3           mprotect
  0.00    0.000000           0         1           munmap
  0.00    0.000000           0         1           brk
  0.00    0.000000           0         2           pread64
  0.00    0.000000           0         1         1 access
```

5.5 Alternative Concept and Complexity Analysis: fopen / fread / fwrite instead of open / read / write

Solution. The C library keeps a buffer (4096 bytes by default for files on this system) inside each FILE object. fread() copies from this buffer and calls read() only when the buffer is empty; fwrite() fills the buffer and calls write() only when it is full. Pseudocode:

fread(p, 1, b, fp): while need bytes: if buffer empty then read(fd, buf, 4096); copy min(avail, need) bytes to p.

Number of system calls for a file of n bytes: direct read() with buffer b gives n / b calls; stdio with its 4096-byte

buffer gives about n / 4096 calls however small the program's own chunk b is. Measured with a 256 KB file (n = 262,144):

Method	Program chunk size	read() calls	write() calls	Formula
read/write direct	1 byte	262,146	262,144	n / b = 262,144
read/write direct	4096 bytes	66	64	n / 4096 = 64 (+2)
fread/fwrite	1 byte	66	64	n / 4096 = 64 (+2)
fread/fwrite	4096 bytes	66	64	n / 4096 = 64 (+2)

The extra 2 reads are the dynamic loader's read of libc and the final end-of-file read. **Space:** direct calls use only the program's own buffer b, O(b); stdio uses b plus a 4096-byte library buffer per open FILE, O(b + 4096) per file, so a program with many open files uses 4 KB more for each. **Time:** T = (calls) x c_{sys} + n x c_{copy}; stdio turns the first term from n / b into n / 4096 for small b at the cost of one extra memory copy. Both are O(n) overall; the difference is in the constant.

Judgement: use stdio for programs that read or write in small pieces; use read() and write() with a large buffer when the program already works in big blocks and wants control over buffering.

TASK 6 - Evaluate the Cost of System Calls

4 marks | K5 Evaluate

6.1 Why a system call costs much more than a function call

A function call is a jump and a return inside the same mode. A system call also changes CPU mode, saves and restores registers, validates arguments, runs kernel code, and may flush or pollute CPU caches and the pipeline. Measured with a loop of 2,000,000 calls: **a function call took about 2 ns and a system call (getppid) about 84 ns, roughly 40 times more**. A real read() or write() costs more still (microseconds), because it also moves data.

6.2 Complete the Code: copy_file() with read() and write()

TODO 1 is completed by a loop that reads up to bufsize bytes, counts every read() call (including the last one that returns 0), stops on 0 or -1, and writes exactly the n bytes read (with an inner loop to handle a short write). The rest of the program is unchanged.

Source: `Task06/copy_file.c`

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of bufsize bytes.
   Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY); if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644); if (out < 0) { close(in); return -1; }
    long calls = 0; ssize_t n;
    /* TODO 1 (completed): loop until read() returns 0 (EOF) or -1; count every read() call;
       write exactly the n bytes that were read */
    while (1) {
        n = read(in, buf, bufsize);
        calls++;
        if (n <= 0) break;
        ssize_t done = 0;
        while (done < n) {                      /* handle a short write */
            ssize_t w = write(out, buf + done, (size_t)(n - done));
            if (w < 0) { close(in); close(out); return -1; }
            done += w;
        }
    }
    close(in); close(out); return calls;
}
int main(void)
{
    FILE *f = fopen("records.dat", "w");
    /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f); fclose(f);

    size_t sizes[] = {1, 512, 4096}; for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i], copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat"); unlink("copy.dat"); return 0;
}
```

Output of the program (matches the expected output) (file `Task06/output.txt`)

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Buffer	Expected	My output	Check
1 byte	10001	10001	10,000 data reads + 1 end-of-file read
512 bytes	21	21	ceil(10000/512) = 20 + 1
4096 bytes	4	4	ceil(10000/4096) = 3 + 1

6.3 strace -c compared with the program's own count

The benchmark program `bench_copy.c` copies a 256 KB file; strace -c counts the calls. For a buffer of b bytes the reads should be n / b + 1 (end of file) plus one read made by the dynamic loader.

Source: `Task06/bench_copy.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
```

```
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
/* usage: bench_copy MODE src dst size   MODE = rw | stdio | mmap */
int main(int argc, char **argv)
{
    if (argc < 5) return 2;
    const char *mode = argv[1]; size_t b = (size_t)atol(argv[4]);
    if (!strcmp(mode, "rw")) {
        char *buf = malloc(b); int in = open(argv[2], O_RDONLY), out = open(argv[3], O_WRONLY|O_CREAT|O_TRUNC, 0644);
        ssize_t n; while ((n = read(in, buf, b)) > 0) { if (write(out, buf, n) != n) return 1; }
        close(in); close(out); free(buf);
    } else if (!strcmp(mode, "stdio")) {           /* fread/fwrite with chunk b, library buffer 4096 default */
        FILE *in = fopen(argv[2], "rb"), *out = fopen(argv[3], "wb"); char *buf = malloc(b); size_t n;
        while ((n = fread(buf, 1, b, in)) > 0) fwrite(buf, 1, n, out);
        fclose(in); fclose(out); free(buf);
    } else {                                        /* mmap: map source, write whole mapping once */
        int in = open(argv[2], O_RDONLY), out = open(argv[3], O_WRONLY|O_CREAT|O_TRUNC, 0644);
        struct stat st; fstat(in, &st);
        void *p = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, in, 0);
        size_t done = 0; while (done < (size_t)st.st_size) { ssize_t w = write(out, (char*)p + done, st.st_size - done); if (w < 0) r
        munmap(p, st.st_size); close(in); close(out);
    }
    return 0;
}
```

strace -c results (file `Task06/output_strace.txt`)

```
--- rw, buffer 1 bytes (256 KB file)
% time     seconds  usecs/call     calls    errors syscall
 50.69    2.019674           7    262144           write
 49.31    1.964493           7    262146           read
--- rw, buffer 512 bytes (256 KB file)
% time     seconds  usecs/call     calls    errors syscall
 51.90    0.000259           0       514           read
 48.10    0.000240           0       512           write
--- rw, buffer 4096 bytes (256 KB file)
% time     seconds  usecs/call     calls    errors syscall
 50.60    0.000592           9        64           write
 49.40    0.000578           8        66           read
--- rw, buffer 65536 bytes (256 KB file)
% time     seconds  usecs/call     calls    errors syscall
 52.78    0.000057           9         6           read
 47.22    0.000051          12         4           write
--- stdio, chunk 1 (256 KB file)
% time     seconds  usecs/call     calls    errors syscall
 51.75    0.000592           9        64           write
 45.54    0.000521           7        66           read
  1.40    0.000016           2         8           mmap
  1.31    0.000015          15         1           munmap
100.00    0.001144           8       139           total
--- stdio, chunk 4096 (256 KB file)
% time     seconds  usecs/call     calls    errors syscall
 51.88    0.000538           8        64           write
 48.12    0.000499           7        66           read
  0.00    0.000000           0         8           mmap
  0.00    0.000000           0         1           munmap
100.00    0.001037           7       139           total
--- mmap, chunk 0 (256 KB file)
% time     seconds  usecs/call     calls    errors syscall
 41.53    0.000076          76         1           write
 38.80    0.000071           7         9           mmap
 15.85    0.000029          14         2           munmap
  3.83    0.000007           7         1           read
100.00    0.000183          14        13           total
```

Buffer b	Expected reads n/b + 1	strace read() count	Writes n/b	strace write() count
1 byte	262,145	262,146	262,144	262,144
512 bytes	513	514	512	512
4096 bytes	65	66	64	64
65,536 bytes	5	6	4	4

The strace figure is exactly one more than the program's own count because the loader reads libc once before main(); this confirms the program's count is right.

6.4 Effect of buffer size on time (8 MB file)

Output of time for each buffer size (bash time: real, user, sys) (file `Task06/output_time.txt`)

```
rw    buffer        1 : 3.162 real  0.676 user  2.478 sys
rw    buffer       16 : 0.205 real  0.053 user  0.150 sys
rw    buffer       64 : 0.062 real  0.004 user  0.052 sys
rw    buffer      512 : 0.017 real  0.006 user  0.006 sys
rw    buffer     4096 : 0.011 real  0.006 user  0.000 sys
```

```
rw    buffer    65536 : 0.008 real  0.004 user  0.000 sys
rw    buffer  1048576 : 0.023 real  0.003 user  0.015 sys
stdio chunk         1 : 0.135 real  0.129 user  0.004 sys
stdio chunk       512 : 0.012 real  0.000 user  0.007 sys
stdio chunk      4096 : 0.010 real  0.000 user  0.006 sys
mmap + write one go   : 0.013 real  0.002 user  0.006 sys
(copy verified identical with cmp)
```

Buffer	read()+write() calls (8 MB)	Real time	Sys (kernel) time	Reading
1 byte	16.8 million	3.16 s	2.48 s	Almost all time is spent entering and leaving the kernel
16 bytes	1.05 million	0.205 s	0.150 s	15x faster than 1 byte
64 bytes	262,144	0.062 s	0.052 s	
512 bytes	32,768	0.017 s	0.006 s	Per-call overhead falling
4096 bytes	4,096	0.011 s	about 0	Matches the page and disk block size
65,536 bytes	256	0.008 s	about 0	Best measured value
1,048,576 bytes	16	0.023 s	0.015 s	Slower: the large buffer no longer fits in the CPU cache and page faults appear

6.5 Library buffering

With fread/fwrite a 1-byte chunk size took 0.135 s (against 3.16 s for read/write with a 1-byte buffer), 23 times faster, because the library issued only 64 reads and 64 writes for a 256 KB file instead of 262,144 of each. Once the program's own chunk reaches 512 or 4096 bytes stdio and direct calls are about equal (0.012 s and 0.010 s).

6.6 Best buffer size for the hospital backup program

Recommendation: 64 KB (65,536 bytes). It gave the lowest measured time (0.008 s for 8 MB), cuts the call count to n / 65,536 (256 calls for 8 MB, a 65,000-fold reduction against 1-byte buffers), and uses little memory. Going to 1 MB made it slower on this machine. A 4 KB buffer is acceptable but needs 16 times more calls.

6.7 Alternative Concept and Complexity Analysis: memory-mapped files (mmap)

Solution. Instead of reading into a buffer, map the whole source file into the address space with one mmap() call; the data are brought in by page faults when touched, and the program writes the mapping to the destination. Code: see mode `mmap` in bench_copy.c above.

Criterion	read()/write() with buffer b	mmap
System calls (measured, 256 KB)	n / b + 2 reads and n / b writes: 66 + 64 for b = 4096	13 in total: 1 write, 9 mmap, 2 munmap, 1 read (loader). Constant, O(1) in the file size
Copying	Kernel to user buffer, then user buffer to kernel again: two copies	The page cache is mapped directly into the process, so the read-side copy is avoided; one copy remains for the write
Memory (space)	O(b), fixed and small	Address space O(n); physical memory only for pages touched, but a very large file (many GB) takes much virtual space and page-table memory, and page faults can stall
Time (8 MB measured)	0.008-0.011 s for 4-64 KB	0.013 s in this test

On this small file mmap was not faster in wall time, because page-fault handling replaces system-call cost; its advantage grows for repeated random access to a large file and when the data are not copied at all. For a straight sequential backup the 64 KB read/write loop is simpler and as fast, so it remains the recommendation.

TASK 7 - Evaluate Process Control System Calls

4 marks | K5 Evaluate

7.1 What fork(), exec(), wait() and exit() do

Call	Action
fork()	Creates a child that is a copy of the parent (same code, a copy-on-write copy of memory, same open files). Returns 0 in the child and the child's pid in the parent, or -1 on failure.
exec() family (execlp)	Replaces the calling process's program with a new one; the pid stays the same. Returns only if it fails.
wait() / waitpid()	Parent blocks until a child ends, then receives its exit status and the kernel releases the child's entry.
exit() / _exit()	Ends the process with a status code; closes files; the parent is notified. exit() also flushes stdio buffers and runs exit handlers, _exit() does not.

7.2 Complete the Code: running jobs like a shell

TODO 1 is completed with fork(); in the child, execlp("expr", "expr", "6", "*", "7", NULL) with exit(127) if exec fails; in the parent waitpid() and printing the status using WEXITSTATUS. The given code uses the variables pid and status, so their declaration is added at the top of main.

Source: `Task07/shell_job.c`

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid; int status;
    printf("shell: running job 1\n"); fflush(stdout);
    /* empty the buffer before fork() */
    /* TODO 1 (completed) */
    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127);                              /* only reached if exec fails */
    }
    waitpid(pid, &status, 0);
    printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n"); fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);
    waitpid(pid, &status, 0);
    /* a child that fails with code 3 */
    printf("job 2 exit status %d\n", WEXITSTATUS(status)); return 0;
}
```

Output (matches the expected output) (file `Task07/output.txt`)

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

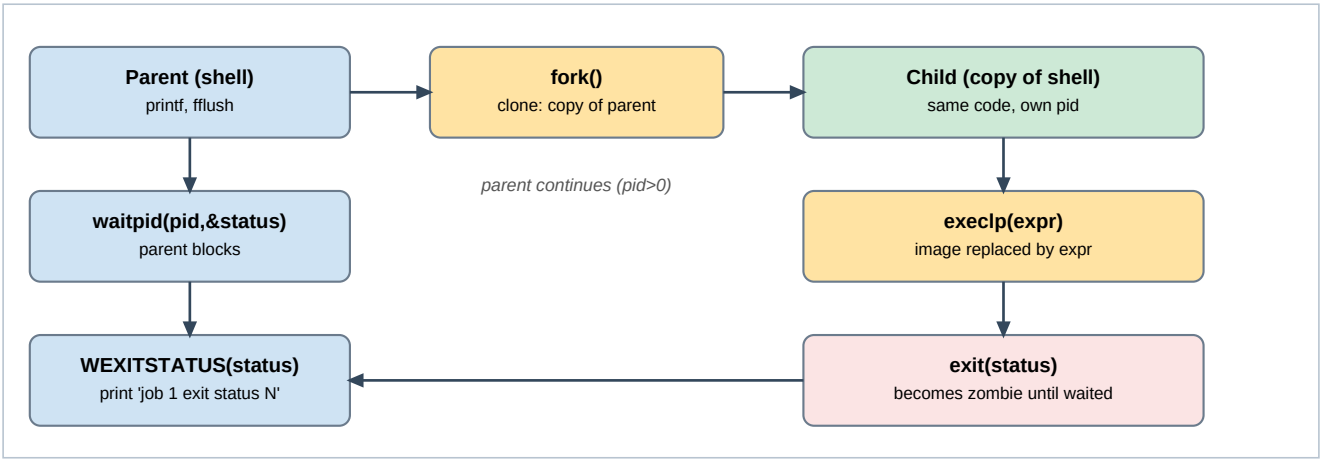


Figure 7.1 - Process diagram for job 1: fork, exec, wait and exit.

Strace of the two jobs shows the same sequence (clone is the system call behind fork; the shell searched the PATH directories before finding /usr/bin/expr):

strace -f -e trace=clone,execve,wait4,exit_group (file `Task07/output_strace.txt`)

```
15828 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7fbae9983a10) = 15829
15828 wait4(15829,  <unfinished ...>
15829 execve("/usr/bin/expr", ["expr", "6", "*", "7"], 0x7ffe59a4c6d8 /* 236 vars */) = 0
15829 exit_group(0)                     = ?
15828 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 15829
15828 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7fbae9983a10) = 15830
15828 wait4(15830,  <unfinished ...>
15830 exit_group(3)                     = ?
15828 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 3}], 0, NULL) = 15830
15828 exit_group(0)                     = ?
```

7.3 Why stdout is flushed before each fork(), and why "job 2" appears twice

When stdout is a terminal it is line-buffered, but when it is redirected to a file or pipe it is **fully buffered**: text waits in a buffer in the process's memory. fork() copies that memory, so a child inherits an *unwritten* copy of the buffer. Each copy is later flushed when its process exits, and the text is printed once by the parent and once by the child. fflush(stdout) before fork() empties the buffer so the child has nothing to repeat.

Experiment: I removed the second fflush(stdout) and changed _exit(3) to exit(3) (file `shell_job_variant.c`). exit() flushes the child's inherited copy of the buffer; _exit() would have thrown it away. Output when redirected:

Output of the variant with stdout not a terminal (file `Task07/output_variant.txt`)

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

The buffer at fork time contained "job 1 exit status 0\nshell: running job 2\n" (the first fflush emptied it earlier, the later printf calls had not been flushed). The child's exit(3) wrote this text again, so **"shell: running job 2" (and "job 1 exit status 0") appear twice**. With the original program (flush before each fork and _exit(3)) every line appears once.

7.4 Running commands with a tiny shell, zombies and orphans

`minishell.c` reads commands, forks, execs, waits and reports each exit status; failures give a non-zero status (2 from ls, 127 when the command is not found).

Source: `Task07/minishell.c`

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
/* Tiny shell: reads one command per line, fork()s, exec()s, wait()s and reports the exit status */
int main(void)
{
    char line[256];
    while (printf("hospital$ "), fflush(stdout), fgets(line, sizeof line, stdin)) {
        line[strcspn(line, "\n")] = 0;
        if (!*line) continue;
        if (!strcmp(line, "exit")) break;
        char *argv[16]; int n = 0; char shown[256]; strcpy(shown, line);
        for (char *t = strtok(line, " "); t && n < 15; t = strtok(NULL, " ")) argv[n++] = t;
        argv[n] = NULL;
        printf("running: %s\n", shown); fflush(stdout);
        pid_t pid = fork();
        if (pid == 0) { execvp(argv[0], argv); perror(argv[0]); _exit(127); }
        int st; waitpid(pid, &st, 0);
        printf("[pid %d finished, exit status %d]\n", pid, WEXITSTATUS(st));
    }
    return 0;
}
```

Output (file `Task07/output_minishell.txt`)

```
hospital$ running: echo hello ward
hello ward
[pid 22719 finished, exit status 0]
hospital$ running: expr 6 + 7
13
[pid 22720 finished, exit status 0]
hospital$ running: ls /nonexistent
ls: cannot access '/nonexistent': No such file or directory
```

```
[pid 22721 finished, exit status 2]
hospital$ running: badcmd
badcmd: No such file or directory
[pid 22722 finished, exit status 127]
hospital$
```

A **zombie** is a child that has ended but whose parent has not yet called wait(); its process-table entry (status code) remains so the parent can collect it, shown as state Z. An **orphan** is a child whose parent ended first; the OS re-parents it to init (pid 1) or a subreaper, which calls wait() when it ends. The program below shows both:

Source: `Task07/zombie_orphan.c`

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
    /* zombie: child exits at once, parent does not wait for 2 s */
    pid_t z = fork();
    if (z == 0) _exit(0);
    sleep(1);
    char cmd[64]; snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,comm -p %d", z);
    printf("zombie child (state Z = defunct):\n"); fflush(stdout); system(cmd);
    waitpid(z, NULL, 0);                        /* parent reaps it */
    printf("after waitpid the zombie is removed:\n"); fflush(stdout);
    system(cmd);
    /* orphan: parent exits before the child; child is re-parented */
    pid_t o = fork();
    if (o == 0) { printf("orphan: my parent is %d before sleep\n", getppid()); fflush(stdout); sleep(1);
                  printf("orphan: my parent is now %d (adopted by init/subreaper)\n", getppid()); fflush(stdout); _exit(0); }
    usleep(200000); return 0;
}
```

Output (file `Task07/output_zombie.txt`)

```
zombie child (state Z = defunct):
  PID  PPID STAT COMMAND
15840 15839 Z    zombie_orphan
after waitpid the zombie is removed:
  PID  PPID STAT COMMAND
orphan: my parent is 15839 before sleep
orphan: my parent is now 1 (adopted by init/subreaper)
```

The zombie (state Z) disappeared after the parent's waitpid(). The orphan's parent changed from the original parent to **1** after the parent ended. Many zombies would exhaust process-table slots, so long-running servers such as the database must reap their children.

7.5 Why fork() followed by exec() rather than one create-process call

- Between fork() and exec() the child can change its own environment: redirect standard input and output to files or pipes, change directory, drop privileges or set limits. This is how a shell implements `cmd > file` and pipelines, using small, general calls.
- A single create-process call would need arguments for every possible setting (as Windows CreateProcess has, with many parameters).
- fork alone is also useful (servers that serve each client in a copy), and exec alone is useful (replace the current program). Two simple primitives compose into many behaviours.

7.6 Alternative Concept and Complexity Analysis: posix_spawn() or threads

Source: `Task07/spawn_bench.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <spawn.h>
#include <pthread.h>
#include <sys/wait.h>
#include <time.h>
extern char **environ;
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return t.tv_sec*1e3+t.tv_nsec/1e6;}
static void *job(void *a){ volatile long s=0; for(int i=0;i<1000;i++) s+=i; return a; }
int main(void)
{
    double t = now();
    for (int i = 0; i < 1000; i++) { pid_t p = fork(); if (p == 0) { execl("/bin/true","true",(char*)NULL); _exit(127);} waitpid(p,NU
    printf("1000 x fork+exec /bin/true : %8.1f ms\n", now()-t);
    t = now();
    for (int i = 0; i < 1000; i++) { pid_t p; char *a[]={"true",NULL}; posix_spawn(&p,"/bin/true",NULL,NULL,a,environ); waitpid(p,NUL
    printf("1000 x posix_spawn /bin/true: %8.1f ms\n", now()-t);
    t = now();
```

```
    for (int i = 0; i < 1000; i++) { pid_t p = vfork(); if (p == 0) { execl("/bin/true","true",(char*)NULL); _exit(127);} waitpid(p,N
    printf("1000 x vfork+exec /bin/true : %8.1f ms\n", now()-t);
    t = now();
    for (int i = 0; i < 1000; i++) { pthread_t h; pthread_create(&h,NULL,job,NULL); pthread_join(h,NULL); }
    printf("1000 x thread (no exec)     : %8.1f ms\n", now()-t);
    return 0;
}
```

Time to start 1,000 commands (/bin/true) (file `Task07/output_spawn.txt`)

```
1000 x fork+exec /bin/true :    777.0 ms
1000 x posix_spawn /bin/true:    678.9 ms
1000 x vfork+exec /bin/true :    693.7 ms
1000 x thread (no exec)     :     35.9 ms
```

Criterion	fork + exec	posix_spawn / vfork + exec	Threads
Time for 1,000 starts (measured)	777 ms	679 ms (spawn), 694 ms (vfork)	36 ms (but they run a function, not a separate program)
Memory copied or shared	Copy-on-write copy of the parent's page tables; cost grows with the parent's size	Does not copy the parent's address space (vfork shares it until exec); cost does not depend on the parent's size	Share all memory with the parent: nothing is copied
Isolation	Full: separate address space	Full once the new program runs	None: one thread's bug can overwrite another's data or crash the whole process

Effect of the parent's size on fork() time (measured) (file `Task10/output_forksize.txt`)

```
parent with    1 MB touched: fork+wait = 0.111 ms each
parent with   64 MB touched: fork+wait = 1.180 ms each
parent with  512 MB touched: fork+wait = 6.103 ms each
```

fork() time rose from 0.11 ms (1 MB parent) to 1.18 ms (64 MB) to 6.1 ms (512 MB) because page tables must be copied: cost O(M), where M is the number of mapped pages. posix_spawn and vfork avoid this, O(1). Starting k commands costs k x (spawn cost + program load), O(k); the exec and program loading dominate, so posix_spawn saved about 13% here, and the advantage grows when the parent is large.

Judgement: keep fork+exec for the shell (flexible redirection); use posix_spawn for a large database server that launches many helper programs; use threads only for cooperating tasks inside one trusted program, never for separate commands that need isolation.

TASK 8 - Evaluate CPU States and Protection

3 marks | K5 Evaluate

8.1 User mode, kernel mode and privileged instructions

The CPU has a **mode bit**: kernel (0) or user (1). Instructions that control the machine (disable interrupts, change page tables, perform I/O, halt, set the timer) are **privileged** and execute only in kernel mode. If a user program tries one, the CPU raises a fault (a trap) and the OS terminates or signals the program. A mode switch to kernel mode happens only through a system call, interrupt or exception, at fixed entry points.

8.2 Interrupts, traps and the timer interrupt

An **interrupt** is an asynchronous signal from hardware (a sensor ready, a disk finished). A **trap** is a synchronous, software-generated transfer (a system call, a divide by zero, a bad address). The **timer interrupt** fires at a fixed rate and returns control to the OS so the scheduler can preempt a running program; without it a looping program could keep the CPU forever. For the monitor the timer drives the periodic sampling and the watchdog.

8.3 Evidence: the hardware refusing a program

Source: `Task08/protection.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <setjmp.h>
static sigjmp_buf jb;
static void h(int s){ siglongjmp(jb, s); }
int main(void)
{
    signal(SIGSEGV, h); signal(SIGILL, h);
    int s;
    if (!(s = sigsetjmp(jb, 1))) { printf("user mode: executing privileged instruction CLI (disable interrupts)...\n"); fflush(stdout
        __asm__ volatile("cli"); printf("CLI succeeded (should not happen)\n"); }
    else printf("  -> CPU raised a protection fault, kernel delivered signal %d (%s); instruction refused\n", s, s==SIGSEGV?"SIGSEGV"
    if (!(s = sigsetjmp(jb, 1))) { printf("user mode: reading kernel/unmapped address 0xffff800000000000...\n"); fflush(stdout);
        volatile char c = *(volatile char *)0xffff800000000000UL; (void)c; }
    else printf("  -> MMU blocked the access, signal %d (SIGSEGV)\n", s);
    if (!(s = sigsetjmp(jb, 1))) { printf("user mode: writing to address 0 (another program's/no memory)...\n"); fflush(stdout);
        *(volatile int *)0 = 1; }
    else printf("  -> MMU blocked the access, signal %d (SIGSEGV)\n", s);
    return 0;
}
```

Output (file `Task08/output_protection.txt`)

```
user mode: executing privileged instruction CLI (disable interrupts)...
  -> CPU raised a protection fault, kernel delivered signal 11 (SIGSEGV); instruction refused
user mode: reading kernel/unmapped address 0xffff800000000000...
  -> MMU blocked the access, signal 11 (SIGSEGV)
user mode: writing to address 0 (another program's/no memory)...
  -> MMU blocked the access, signal 11 (SIGSEGV)
```

The privileged instruction CLI (clear interrupt flag) was refused with a protection fault, and the two accesses to forbidden addresses were stopped by the memory-management unit. In each case the kernel received the fault and delivered SIGSEGV to the program, so the rest of the system was unaffected.

8.4 What if any program could disable interrupts?

A faulty or malicious program could execute CLI and loop; no sensor, timer or network interrupt would be handled. The alarm interrupt would never run, the timer could never preempt the program, the watchdog could not fire and the monitor would silently stop alarming while looking normal. This is why disabling interrupts is privileged. Even the kernel disables interrupts only for a few microseconds in critical sections, as this adds directly to the worst-case interrupt latency.

8.5 How the hardware stops a program reading another program's memory

Every address a program uses is a *virtual* address translated by the MMU through that process's page table. A page-table entry holds the physical frame and permission bits (read, write, execute, user/supervisor). Only the kernel can load the page-table base register. Another process's pages are simply not in the page table, and kernel pages are marked supervisor-only, so an access causes a page fault and the kernel stops the program (the second and third tests above). Simple embedded CPUs without an MMU use a memory-protection unit (MPU) with fixed regions.

8.6 Alternative Concept and Complexity Analysis: polling instead of interrupts

Solution. The monitor's main loop checks the sensor repeatedly (`while (!ready()) ;`) or every T milliseconds, instead of sleeping until the sensor interrupts. I measured both while waiting 2 seconds for an event:

Source: `Task08/poll_vs_interrupt.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <time.h>
#include <sys/resource.h>
#include <sys/select.h>
#include <fcntl.h>
/* Wait 2 s for a "sensor event". Polling: spin on a non-blocking pipe. Interrupt style: block in select() (CPU sleeps). */
static double cpu(void){ struct rusage r; getrusage(RUSAGE_SELF,&r); return r.ru_utime.tv_sec+r.ru_stime.tv_sec+(r.ru_utime.tv_usec+r
static double wall(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
int main(void)
{
    int p[2]; pipe(p); fcntl(p[0], F_SETFL, O_NONBLOCK);
    if (fork() == 0) { sleep(2); write(p[1], "!", 1); _exit(0); }
    char c; long polls = 0; double c0 = cpu(), w0 = wall();
    while (read(p[0], &c, 1) < 0) polls++;
    printf("POLLING  : event noticed after %.2f s wall, CPU used %.2f s, %ld polls\n", wall()-w0, cpu()-c0, polls);
    int q[2]; pipe(q);
    if (fork() == 0) { sleep(2); write(q[1], "!", 1); _exit(0); }
    c0 = cpu(); w0 = wall(); fd_set s; FD_ZERO(&s); FD_SET(q[0], &s);
    select(q[0]+1, &s, NULL, NULL, NULL); read(q[0], &c, 1);
    printf("INTERRUPT: event noticed after %.2f s wall, CPU used %.2f s (blocked until woken)\n", wall()-w0, cpu()-c0);
    return 0;
}
```

Output (file `Task08/output_poll.txt`)

```
POLLING  : event noticed after 2.00 s wall, CPU used 2.00 s, 16696113 polls
INTERRUPT: event noticed after 2.00 s wall, CPU used 0.00 s (blocked until woken)
```

Criterion	Polling	Interrupts
Worst-case delay before an abnormal reading is noticed	Up to one polling interval T (for T = 20 ms the delay can exceed the 10 ms deadline; a 1 ms interval gives up to 1 ms), O(T). Busy-loop polling has near-zero delay	Interrupt latency, typically a few microseconds, independent of how rare events are, O(1)
CPU time wasted (measured, 2 s wait)	2.00 s of CPU: 100% busy, 16.7 million useless checks	0.00 s: the CPU slept until the event
Power	CPU never idles; battery or heat problem; 500 monitors all spinning	CPU enters low-power state between events
Complexity	Simple code, no interrupt handler, predictable timing	Needs handler, priorities, careful shared-data handling

CPU utilisation for polling at period T with check cost c is c / T for timed polling (small if T is large, but delay grows) and 100% for busy-waiting. Interrupts use c_{isr} only when an event occurs, so utilisation is proportional to the event rate. Polling is sometimes right when events are very frequent (a device that always has data), where interrupt overhead exceeds the polling cost.

Judgement: bedside monitors use interrupt-driven sensing with a timer-driven task, which meets the 10 ms alarm with minimal power; polling is rejected.

TASK 9 - Evaluate the Memory Hierarchy and I/O Systems

4 marks | K5 Evaluate

9.1 Levels of the memory hierarchy

Level	Typical size	Typical access time	Measured here
Registers	Under 1 KB	Under 1 ns (about 1 clock cycle)	-
L1 / L2 / L3 cache	32 KB / 256 KB-1 MB / 8-64 MB	1 ns / 3-5 ns / 10-40 ns	1.3 ns (16 KB), 3.2 ns (128 KB), 5.2 ns (1 MB), 31.7 ns (8 MB)
Main memory (RAM)	4 GB-1 TB	60-150 ns	133 ns (64 MB), 167 ns (256 MB)
SSD	256 GB-8 TB	50-150 us	-
Hard disk	1-20 TB	5-10 ms	-
Network storage	Unlimited	0.2-10 ms or more per request	-

Source: `Task09/chase.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <fcntl.h>
#include <unistd.h>
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return t.tv_sec*1e9+t.tv_nsec;}
int main(void)
{
    puts("Memory hierarchy: random pointer-chase, average time per access");
    size_t kb[] = {16, 128, 1024, 8192, 65536, 262144};
    for (int k = 0; k < 6; k++) {
        size_t n = kb[k]*1024/sizeof(void*); void **a = malloc(n*sizeof(void*)); size_t *idx = malloc(n*sizeof(size_t));
        for (size_t i = 0; i < n; i++) idx[i] = i;
        for (size_t i = n-1; i > 0; i--) { size_t j = rand() % (i+1); size_t t = idx[i]; idx[i] = idx[j]; idx[j] = t; }
        for (size_t i = 0; i < n; i++) a[idx[i]] = &a[idx[(i+1)%n]];
        void **p = &a[idx[0]]; size_t steps = 20000000; double t = now();
        for (size_t i = 0; i < steps; i++) p = *p;
        double ns = (now()-t)/steps; printf("  working set %7zu KB : %6.2f ns/access%s\n", kb[k], ns, p?"":"!");
        free(a); free(idx);
    }
    puts("Record look-up: 100,000 records x 64 bytes, 300,000 random look-ups");
    size_t N = 100000; char *mem = malloc(N*64); for (size_t i = 0; i < N*64; i++) mem[i] = i;
    int fd = open("records.bin", O_RDWR|O_CREAT|O_TRUNC, 0644); write(fd, mem, N*64);
    char rec[64]; volatile long sum = 0; srand(1); double t = now();
    for (int i = 0; i < 300000; i++) { size_t r = rand() % N; pread(fd, rec, 64, r*64); sum += rec[0]; }
    double d1 = (now()-t)/300000; srand(1); t = now();
    for (int i = 0; i < 300000; i++) { size_t r = rand() % N; sum += mem[r*64]; }
    double d2 = (now()-t)/300000;
    printf("  pread() per look-up (OS page cache, one system call each): %8.1f ns\n", d1);
    printf("  in-memory cached copy in the program                     : %8.1f ns   (%.0fx faster)\n", d2, d1/d2);
    unlink("records.bin"); return 0;
}
```

Output (file `Task09/output.txt`)

```
Memory hierarchy: random pointer-chase, average time per access
  working set      16 KB :    1.30 ns/access
  working set     128 KB :    3.20 ns/access
  working set    1024 KB :    5.19 ns/access
  working set    8192 KB :   31.68 ns/access
  working set   65536 KB :  133.18 ns/access
  working set  262144 KB :  166.78 ns/access
Record look-up: 100,000 records x 64 bytes, 300,000 random look-ups
  pread() per look-up (OS page cache, one system call each):    237.8 ns
  in-memory cached copy in the program                     :     22.5 ns   (11x faster)
```

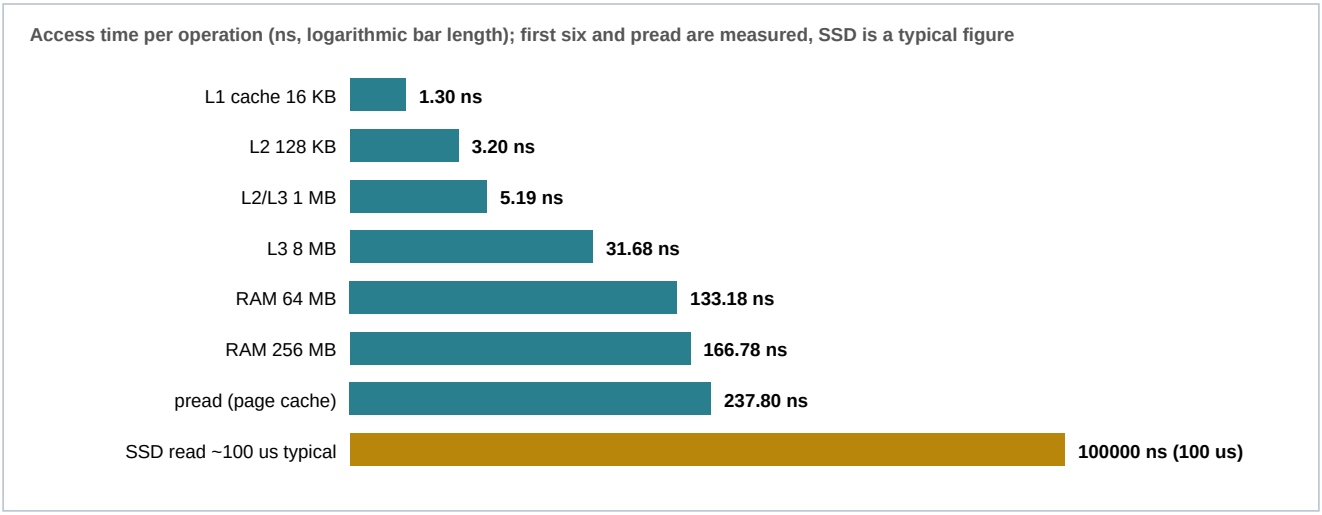


Figure 9.1 - Each step down the hierarchy is slower; the measured steps (cache sizes) are visible in the pointer-chase results.

9.2 Where to keep each kind of data

Data	Placement	Reason
Monitor readings (live)	RAM / on-chip memory of the monitor, short ring buffer; periodic copy to the database	Needs microsecond access and no mechanical or network delay in the alarm path; history is not needed to raise an alarm
Patient records	Database on redundant SSD storage with an in-memory cache of recently used records; backups on disk or network storage	Random look-ups need low latency (SSD 100 us vs disk 5-10 ms); durability needs replication and backup
MRI images	Large disk or network storage (cheap per GB), loaded into RAM in pieces for processing; archive older scans on slower storage	Each scan is about 500 MB, too large to keep thousands in RAM; sequential access works well on disk or SSD
Working data of reconstruction	RAM and CPU caches of the imaging server	Needs the highest bandwidth

9.3 Programmed I/O, interrupt-driven I/O and DMA

Method	How it works	CPU involvement	Suits
Programmed I/O	The CPU loops checking the device status and moves each word	100% while waiting	Very simple or fast devices
Interrupt-driven I/O	The device interrupts when ready; the CPU moves one word per interrupt	Free between interrupts, but one interrupt per word	Keyboards, the monitor's sensors, small transfers
DMA	The CPU programs the DMA controller with address and count; the controller moves the block to memory by itself and interrupts once at the end	Two set-up commands and one interrupt per block	MRI scanner, disks, network cards: bulk transfers

For the MRI scanner: DMA, because the volume is huge and the CPUs should process images, not copy words.

9.4 Effect of caching on record look-up

Effective access time EAT = h x t_{cache} + (1 - h) x t_{slow}. With a hit ratio of 95%, cache time 22 ns and slow path 238 ns, EAT = 0.95 x 22 + 0.05 x 238 = 32.8 ns, against 238 ns without a cache (7 times faster). Measured in `chase.c`: a look-up through pread() (system call plus page cache) took 237.8 ns, while a look-up in the program's own in-memory copy took 22.5 ns, **11 times faster**. From disk the difference is far larger (about 100 us from SSD, i.e. thousands of times slower than memory), so the database keeps recently used patient records in a RAM cache.

9.5 Alternative Concept and Complexity Analysis: interrupt-driven I/O (one interrupt per word) instead of DMA

Solution. The CPU handles every word: the device raises an interrupt, the handler reads the word from the device register and stores it in memory. For a 500 MB image (524,288,000 bytes) with a 4-byte word and an assumed 2 us for each interrupt (entry, handler, return; typical 1-5 us):

Quantity	Interrupt per word	DMA (one interrupt per 64 KB block)
Number of interrupts	524,288,000 / 4 = **131,072,000**	524,288,000 / 65,536 = **8,000**
CPU time for interrupts	131,072,000 x 2 us = **262 s**	8,000 x 2 us = **16 ms**
Interrupt rate needed to keep up with a 1 GB/s device	About 262 million per second: impossible (a CPU handles about 0.5-1 million per second), so data would be lost	About 16,000 per second: easily handled
Memory buffers (space)	Device data register (4 bytes); the CPU stores each word directly into the 500 MB destination image	The same 500 MB destination plus a small descriptor and one or two (double-buffered) 64 KB transfer buffers or a descriptor ring
Time complexity	O(n) interrupts for n bytes	O(n / block) interrupts, the data copy itself is done by hardware

The transfer itself at 1 GB/s needs only 0.5 s; with interrupts per word the CPU would need 262 s, about 500 times longer, leaving no CPU time for image processing. DMA also lets the CPU work on one block while the next arrives.

Data-placement and I/O plan: monitors: RAM ring buffer, interrupt-driven; database: SSD plus RAM cache, DMA; imaging: disk or network storage, DMA with double buffering; workstations and tablets: local cache of the records in use, interrupt-driven I/O for input devices.

TASK 10 - Measure and Compare

4 marks | K5 Evaluate

10.1 Test environment

Item	Value
Compiler	gcc 13.3.0, flags -std=c11 -Wall -Wextra -pthread -O1
OS	Ubuntu 24.04.5 LTS, Linux kernel 6.18
Hardware	Intel Xeon (virtual machine), 2 vCPUs, 8 GB RAM
Method	Each program run once for tables below (fork/thread test run repeatedly with similar results: 98.9, 97.9 and 93.6 ms for processes; 28.6-31.1 ms for threads)

10.2 System calls for different buffer sizes (strace -c, 256 KB file)

Buffer size	read() calls	write() calls	Total I/O calls	Time in strace
1 byte	262,146	262,144	524,290	4.0 s
512 bytes	514	512	1,026	about 0.0005 s
4096 bytes	66	64	130	about 0.001 s
65,536 bytes	6	4	10	about 0.0001 s

Totals follow n / b: the number of calls fell by a factor of 4,096 from 1 byte to 4 KB buffers and 65,536 from 1 byte to 64 KB buffers.

10.3 User time versus kernel time (time command, 8 MB copy)

Buffer	Real (s)	User (s)	Sys (s)	Where the time goes
1 byte	3.162	0.676	2.478	78% kernel: 16.8 million kernel entries
16 bytes	0.205	0.053	0.150	73% kernel
64 bytes	0.062	0.004	0.052	84% kernel
512 bytes	0.017	0.006	0.006	Overhead nearly gone
4096 bytes	0.011	0.006	0.000	Mostly copying data
65,536 bytes	0.008	0.004	0.000	Fastest
1 MB	0.023	0.003	0.015	Cache and page-fault costs return
fread/fwrite, 1-byte chunks	0.135	0.129	0.004	Time is in library code in user mode instead
mmap + one write	0.013	0.002	0.006	Page faults instead of calls

10.4 Creating 1,000 processes and 1,000 threads

Source: `Task10/create_bench.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <time.h>
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return t.tv_sec*1e3+t.tv_nsec/1e6;}
static void *th(void *a){ return a; }
int main(void)
{
    double t = now();
    for (int i = 0; i < 1000; i++) { pid_t p = fork(); if (p == 0) _exit(0); waitpid(p, NULL, 0); }
    double fp = now()-t;
    t = now();
    for (int i = 0; i < 1000; i++) { pthread_t h; pthread_create(&h, NULL, th, NULL); pthread_join(h, NULL); }
    double ft = now()-t;
    /* creation of 1000 live threads at once vs 1000 live processes: peak memory */
    struct rusage r0; getrusage(RUSAGE_SELF, &r0);
    printf("1000 processes (fork+wait)   : %7.1f ms (%.3f ms each)\n", fp, fp/1000);
    printf("1000 threads (create+join)   : %7.1f ms (%.3f ms each)  -> processes %.1fx slower\n", ft, ft/1000, fp/ft);
    return 0;
}
```

Output (file `Task10/output_create.txt`)

```
1000 processes (fork+wait)   :    93.6 ms (0.094 ms each)
1000 threads (create+join)   :    30.7 ms (0.031 ms each)  -> processes 3.0x slower
```

Memory of 200 live threads (file `Task10/output_mem.txt`)

```
200 threads : process RSS 9612 KB -> 11708 KB (+2096 KB total, shared heap not copied)
```

Item	1,000 processes (fork)	1,000 threads	Explanation
Creation + join time	93.6 ms (0.094 ms each)	30.7 ms (0.031 ms each), 3.0x faster	fork must duplicate page tables and process structures; a thread shares the address space and file table, needing only a stack and a thread descriptor
Memory	Each process has its own page tables and writes cause private copies (copy-on-write); at 64 MB parent size fork cost rose 10x	200 threads added only 2,096 KB in total (about 10 KB each) with the 8 MB heap shared	Threads share code, heap, and files; only stack and registers are private
Isolation	A crash stays inside the process	A crash or stray write affects every thread	What is shared is also what is exposed

10.5 How the measurements support or contradict my earlier judgements

Earlier judgement	Measurement	Verdict
System calls are far more expensive than function calls (Task 6)	84 ns against 2 ns, about 40x	Supported
Larger buffers reduce calls and time (Task 6)	Time fell from 3.16 s to 0.008 s as buffers grew from 1 byte to 64 KB	Supported
Very large buffers are always better	1 MB was 2.9x slower than 64 KB	**Contradicted**: there is an optimum, because of cache and page-fault effects
mmap should be faster than read/write (Task 6)	0.013 s against 0.008-0.011 s	**Not confirmed** on a small sequential copy: page faults replace calls; benefit appears for large random access
Threads are cheaper than processes (Task 7, 10)	3.0x faster creation	Supported, but isolation is lost
General-purpose scheduler cannot guarantee 10 ms (Task 1)	Worst wake-up delay about 8 ms under load, 1 ms idle	Supported, and the margin is thin
Message passing costs more than a system call (Task 3)	237x here	Supported; real microkernels do better, but the cost is still higher
Polling wastes CPU (Task 8)	2.00 s of 2.00 s against 0.00 s	Supported

10.6 Alternative Concept and Complexity Analysis: threads instead of processes

Result. Creating 1,000 threads took about 31 ms against about 94 ms for 1,000 processes (3.0 times faster) and the threads added about 10 KB each, because a new thread shares the address space, open files and heap with its creator and only needs its own stack and registers. A process creation also has to set up a new address space: the page-table copy cost grows with the parent size (0.11 ms for a 1 MB parent, 6.1 ms for a 512 MB parent). Thread creation is O(1) in the parent size; fork is O(M). Space: threads O(stack size) each; processes O(page tables) + any pages written after fork. The difference is explained entirely by what is shared.

TASK 11 - Recommend and Defend the Operating-System Plan

4 marks | K5 Evaluate

11.1 The complete plan

Environment	OS type	Structure	Services	Protection	I/O method
Bedside monitor	Hard real-time (FreeRTOS / QNX)	Small kernel / microkernel	Scheduler, interrupts, timer, sensor driver, secured channel, watchdog	MPU/mode bit, signed firmware, minimal code	Interrupt-driven; timer task
Nurse tablet	Handheld (Android)	Layered over modular Linux	Touch UI, comms, power management, encrypted cache	App sandbox, encryption, remote wipe	Interrupt-driven; battery aware
Doctor workstation	Desktop (Linux or Windows)	Modular / hybrid	GUI, file system, comms, accounting	Login, ACLs, audit	Interrupt-driven, DMA for disk
Database server	Clustered Linux with fail-over	Monolithic with modules	Journaling file system, caching, replication, audit	User IDs, ACLs, encryption, heartbeat monitoring	DMA, RAM cache, SSD
Imaging server	SMP Linux	Monolithic with modules	Process and memory management, large file I/O	Restricted accounts	DMA with double buffering
Partner link	Distributed system over secured network	User-space network services	Communication, authentication, audit	Mutual TLS, encryption, firewalls	Network DMA

11.2 Evidence supporting the plan

- **Alarm timing:** a general-purpose scheduler showed worst-case wake-up of about 8 ms under load, leaving almost nothing of the 10 ms budget, so the monitor uses a hard real-time OS (Task 1 measurement).
- **Fault containment:** a microkernel request costs more (about 20 us here) but 20 us is 0.2% of the deadline, and a faulty driver cannot crash the alarm (Task 3).
- **Database availability:** a two-node cluster gives 99.9975% (about 13 minutes of downtime a year) against 99.9% (8.76 hours) for one large machine; rolling updates remove update downtime (Task 2).
- **Tablets:** a reduced OS saves about 4x storage and roughly doubles battery life compared with a full desktop OS (typical figures, Task 4).
- **Backup program:** 64 KB buffers cut system calls by 65,000 times and time from 3.16 s to 0.008 s for 8 MB (Tasks 6 and 10).
- **Imaging:** DMA needs 8,000 interrupts for a 500 MB scan instead of 131 million, and SMP gives up to 12.5x on 32 CPUs with 95% parallel work (Tasks 2 and 9).
- **Protection:** mode bit, memory protection and file permissions were demonstrated stopping a privileged instruction, wrong memory access and an unauthorised read (Tasks 4 and 8).

11.3 Limitations and risks

Risk	Impact	Mitigation
Five different OS families increase administration and skill requirements	Higher cost; slower patching	Standardise on Linux for servers and workstations; one RTOS family; central device management
RTOS guarantee holds only if the application obeys the design (priority mistakes, long interrupt handlers)	Missed alarm	Timing analysis, test with worst-case load, watchdog
Cluster fail-over can fail (split brain, shared-storage fault)	Database stop	Quorum or witness node, tested fail-over drills, backups
My timing measurements come from a virtual machine and typical figures are not measured	Absolute values may differ on real hardware	Repeat on the target hardware before purchase
SMP scaling is limited by the serial part	Imaging speed not linear	Add a second imaging server rather than more CPUs

Risk	Impact	Mitigation
Security of the partner link depends on key management	Data breach	Mutual TLS, key rotation, audit logs

11.4 Suitability for a group of three hospitals

The plan scales: each hospital repeats the same monitor, tablet, workstation, database-cluster and imaging pattern and joins one distributed layer for shared records. Monitors, tablets and workstations scale by adding identical devices (500 + 120 + 80 per hospital, about 3 times for three). The database must change from one cluster per hospital to replicated clusters with a consistency protocol, and the partner link becomes a mesh of three secured links; both are extensions of the distributed design, not a new architecture. Task 12 shows the further extension to home devices and cloud records.

11.5 Alternative Concept and Complexity Analysis: everything in virtual machines on two large servers

Solution. Host the database, imaging, workstation desktops and partner services as virtual machines (VMs) on two large servers with live migration. A hypervisor shares CPUs, memory and I/O among the VMs.

Criterion	Separate physical machines	VMs on two large servers
Response time for monitors	Monitors run on their own hardware: bounded latency	If monitor services were in a VM, the hypervisor adds scheduling and interrupt-virtualisation delay. My own tests run inside a VM and still showed 0.3-2 ms spikes even at real-time priority, so jitter is real. The bedside monitors must stay on physical RTOS devices; only back-end services move
Processor overhead	None	Typically about 2-10% CPU for compute work, more for I/O-heavy work unless pass-through or paravirtual drivers are used (typical)
Memory overhead	None	Each VM carries its own guest OS (about 0.5-2 GB) plus hypervisor overhead; sharing identical pages recovers part; total memory O(number of VMs x guest OS size)
Reliability	A failure affects one function	Two servers: a server fault takes down about half the VMs, restart or migration takes seconds to minutes; availability $1 - (0.001)^2$ is 99.9999% only if every VM can fit on one server; the hypervisor adds a new single failure layer and a new security layer. Fewer machines to maintain
Cost and administration	More hardware, more power	Fewer machines, snapshots, easy rollback, easier updates (advantage)

Judgement: virtualise the back-end (database replicas, imaging pre-processing, partner services, workstations as virtual desktops) to save cost, but keep the bedside monitors on dedicated real-time devices and keep the database as a cluster across both servers with replicas on a third site.

TASK 12 - Re-evaluate and Adapt the Design (Day-4 Design Challenge)

8 marks | K5 Evaluate

12.0 The original design (starting point)

Task 12 re-evaluates my plan from Tasks 1-11. The table summarises it so the changes can be traced.

Environment	OS type	Structure	Key decisions
Bedside monitors (500)	Hard real-time OS (FreeRTOS/QNX)	Small kernel / microkernel	Interrupt-driven sensing, fixed-priority scheduling, 10 ms alarm deadline, no UI or general file system
Nurse tablets (120)	Handheld OS (Android, Linux kernel)	Modular monolithic	Power management, touch UI, app sandboxing, battery lasts a shift
Doctor workstations (80)	Desktop time-sharing OS	Monolithic / hybrid	Full UI, file system, user accounts, accounting
Database server	Clustered Linux, 2-node fail-over	Monolithic with loadable modules	Fail-over within seconds, journaling file system, ACLs
Imaging server	Symmetric multiprocessing (SMP) Linux	Monolithic with modules	DMA transfers for MRI data, many CPUs share one kernel
Partner-hospital link	Distributed system	Network services in user space	Point-to-point secured link between two partner hospitals

Original assumption that now breaks: everything sat inside hospital buildings on a trusted, bounded-latency network under one administrator.

12.1 Impact Analysis (2 marks)

Two changes arrive together. **Home devices** add a new environment that the hospital does not physically control. **Cloud records** move the system of record out of the hospital's own database cluster and make it shared by three organisations. The table lists what is affected, using the categories the rubric asks for.

Component	Affected?	OS type / service / protection impact
Home monitors (NEW)	New	Needs an **embedded hard real-time OS**. Services: program execution, I/O, communication, error detection, protection. Protection: secure boot, memory-protection unit, device identity. Runs on battery or mains in an uncontrolled home.
Cloud records service (NEW)	New	**Distributed system** on Linux hosts (virtual machines or containers). Services: communication, file system (replication), resource allocation, accounting, protection.
Database cluster	Modified	Becomes a local replica of the cloud record. **Cluster fail-over** is kept for local continuity but is no longer the only copy. The file-system and consistency service must now handle replication and WAN outages.
Partner link	Modified	Point-to-point link is replaced by access to the shared cloud. **Protection** moves from 'trusted link' to mutual authentication, encryption and per-hospital access control.
Nurse tablets	Modified	Same handheld OS, but **protection and communication** change: always-on VPN or mutual TLS, encrypted offline cache, remote wipe.
Doctor workstations	Modified (light)	Same desktop OS. Protection: single sign-on, multi-factor login, audit logging for access to shared records.
Hospital bedside monitors	Retained	Requirements unchanged: local 10 ms alarm, interrupt-driven. Not affected by cloud or home devices.
Imaging server	Retained	SMP and DMA still best. MRI files stay on site. Only reports and metadata go to the cloud.

Protection mechanisms affected. The hardware protections (user/kernel mode bit, memory protection, privileged instructions) still protect each individual machine, but they cannot protect data once it crosses a network or a tenant boundary. Three new mechanisms are needed: (1) **authentication** of every device and user (certificates, not IP addresses); (2) **encryption** in transit and at rest with managed keys; (3) **tenant isolation** and audit trails, so one hospital's staff cannot read another's patients without authorisation.

Conflicts introduced. Fast response (10 ms) conflicts with the internet, whose round-trip delay is unbounded. Shared records conflict with strict security. Long battery life conflicts with constant encrypted transmission.

12.2 Evaluation of the Original Design (2 marks)

I tested the original plan against the three criteria the rubric names. Each verdict is backed by a figure or a reasoned calculation.

Criterion	Evidence / calculation	Verdict on original plan
Response time	**In hospital:** interrupt-driven RTOS on the monitor meets 10 ms because the alarm never leaves the device and the scheduler is fixed-priority (bounded). **At home:** internet round-trip is typically tens of milliseconds and has no upper bound (jitter, loss, retransmission). A cloud-decided alarm therefore cannot promise 10 ms. My measurement on Linux (Task 6): a system call costs about 84 ns against 2 ns for a function call (about 40x), so extra kernel crossings in a general OS add up, but the dominant risk is the network, not the CPU.	**Partly supported.** Valid inside the hospital. Not valid for home devices unless the alarm is made local.
Reliability	Two-node cluster with independent 99% nodes: availability = 1 - $(0.01)^2$ = **99.99%**. It survives a node failure in seconds but **not a site failure** (power, fire, network cut), and the shared cloud record adds a WAN dependency. Three replicas in separate zones: 1 - $(0.01)^3$ = **99.9999%** (assuming independent failures). Home internet outages are common, so devices must buffer readings.	**Partly supported.** Good for node failure; no protection against site loss, WAN partition or home outage.
Security	Original trust model: trusted LAN, one administrator, a point-to-point link. Now: devices in homes (physically reachable by anyone), the public internet, three organisations and a cloud provider. Kernel/user mode and file permissions do not defend against stolen credentials, a tampered device or intercepted traffic.	**Not supported.** Needs zero-trust controls: mutual TLS, signed firmware, encryption, per-tenant access control.
Throughput / scale	Illustrative load: 1,000 home devices x 1 reading per 10 s x 200 bytes = 100 messages/s = 20 KB/s. This is trivial for a modest ingest cluster. Imaging: a 500 MB MRI at 100 Mbit/s takes 500 x 8 / 100 = **40 s** per scan to upload, so images should stay on site.	**Supported** for readings. Imaging must stay local.

Conclusion of the evaluation. The original plan is sound for the parts that stay inside the hospital, but it cannot support home devices and cloud records as it stands. The gaps are in **where the alarm decision is made**, **how many sites hold the data** and **how identity is proven**. None of these gaps requires replacing the monitors, workstation OS or imaging server, so a full redesign is not justified.

12.3 Adaptation: the revised plan (2 marks)

Design principle: **decide locally, replicate globally**. Anything safety-critical is decided on the device by a hard real-time OS. The cloud is used for storage, sharing, trends and non-urgent alerts.

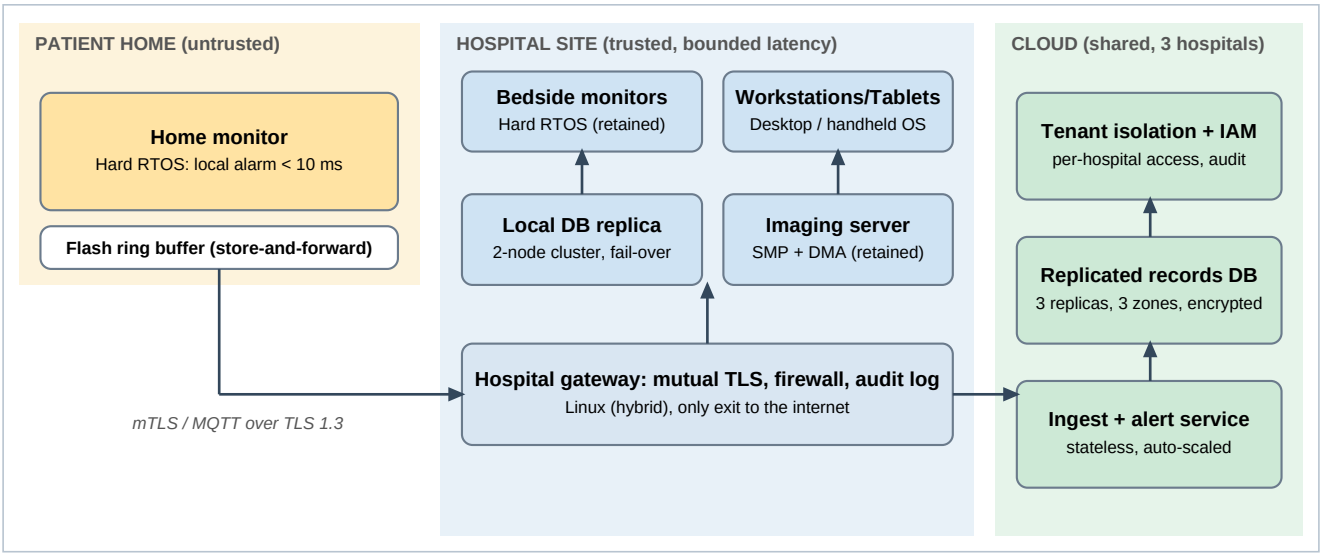


Figure 1 - Revised architecture. The alarm decision stays inside the home device and the bedside monitor; only readings and non-urgent alerts cross the network.

Environment	OS type	Structure	Services kept / removed	Protection and I/O
Home monitor (new)	Embedded hard real-time OS (FreeRTOS with MPU, or Zephyr; QNX if certification needs it)	Small kernel / microkernel; drivers in user space where possible	Keep: scheduling, I/O, networking, error detection, watchdog. Remove: UI, general file system, accounting, multi-user login.	Secure boot, signed A/B firmware updates with rollback, per-device certificate, MPU isolates tasks. Interrupt-driven sensing; timer sleep between samples to save power.
Cloud records (new)	Distributed system on Linux hosts	Monolithic Linux host with modules, running containers/VMs per service	Keep: communication, replicated file system/database, resource allocation, accounting, protection. Remove: local UI.	Mutual TLS, encryption at rest, per-hospital tenant isolation, role-based access, audit log. DMA-capable network and storage I/O.
Database server (modified)	Clustered Linux, now a local replica	Unchanged	Add: replication and conflict handling; keep fail-over.	Keeps ACLs. Reads and writes continue on site during a WAN outage and sync afterwards.
Partner link (modified)	Distributed access to the shared cloud	User-space network services	Replace point-to-point link by cloud API; keep VPN as backup path.	Mutual authentication, per-hospital authorisation.
Tablets (modified)	Handheld OS (unchanged)	Unchanged	Add device management, offline encrypted cache.	Always-on VPN or mTLS, remote wipe, screen lock.
Workstations (modified)	Desktop OS (unchanged)	Unchanged	Add single sign-on and audit.	Multi-factor login.
Monitors, Imaging (retained)	Hard RTOS; SMP Linux	Unchanged	Unchanged.	Unchanged. MRI data stays on site.

Alarm path and latency budget (design targets)

Path	Where decided	Target	Why this OS choice
Local alarm on home device (buzzer, light, local alert)	On the device, in the RTOS interrupt handler and a high-priority task	< 10 ms, guaranteed	Hard real-time: bounded interrupt latency and fixed-priority scheduling. No network involved.
Remote notification to the care team	Cloud ingest and alert service	about 1-5 s, best effort (soft real-time)	Internet delay is unbounded, so a hard deadline cannot be promised. Device keeps retrying and buffering.
Reading storage and sharing	Replicated cloud DB; local replica	Seconds to minutes	Eventual delivery is acceptable because the alarm was already raised locally.

12.4 Evidence-Based Defence (2 marks)

12.4.1 Why MODIFY, rather than RETAIN or REDESIGN

I scored the three options against five criteria. Weights reflect patient safety first. Scores (1 = poor, 5 = excellent) are my judgement from the Section 12.2 findings.

Criterion	Weight	Retain	Modify	Redesign
Patient-safety alarm	30%	2	5	5
Reliability / availability	25%	2	4	4
Security	20%	1	4	4
Cost and risk of change	15%	5	4	2
Scalability	10%	2	4	5
Weighted total (out of 5)	100%	**2.25**	**4.30**	**4.10**

Retain scores 2.25 because it fails safety, security and reliability for the new devices. Modify (4.30) and Redesign (4.10) are close; Modify wins because the monitors, imaging server and workstations meet their requirements today. Redesigning them would add cost and risk with no clinical benefit. If cost and risk were weighted at only 5%, Redesign would pull level, so the decision is sensitive to that weight; I accept this because replacing validated safety-critical software is itself a patient-safety risk.

12.4.2 Evidence from measurement and analysis

Claim in the design	Evidence	Meaning for the decision
Kernel crossings are costly, so the home OS must be small and avoid unnecessary calls	Measured on Linux (gcc 13, 2 CPUs; Tasks 6 and 10): system call about **83-84 ns**, function call about **2 ns** (about 40x) (Task 6).	Supports a lean RTOS for monitors and a general OS only where needed.
Processes cost more than threads, so cloud services scale with lightweight workers, but isolation per tenant needs processes or containers	Measured: **1,000 fork()+wait = about 94 ms** (0.094 ms each); **1,000 thread create+join = about 31 ms** (0.031 ms each). Threads are about 3.0x faster because they share the address space (Task 10).	Use threads inside a service for speed; use separate processes/containers between hospitals for isolation.
Alarm must be local	Availability and latency arguments in Section 12.2; internet delay has no upper bound.	Hard deadline can only be given where the scheduler is bounded.
Imaging stays on site	500 MB over 100 Mbit/s = 40 s per scan.	Send only reports and pointers to the cloud.
Replicate records across zones	99% node, 2 nodes = 99.99%; 3 nodes = 99.9999%.	Local fail-over cluster plus cloud replicas gives site-level tolerance.

Measurements were taken on a Linux system in a cloud container; absolute values vary with hardware, and the ratios, not the exact figures, support the argument. They are the same measurements reported in Tasks 6 and 10.

12.4.3 Trade-offs accepted, risks and mitigations

Trade-off / risk	Impact	Mitigation
Local alarm at home gives no central guarantee of a hard deadline for the remote care team	Remote staff may see the alert seconds later	Local alarm for the patient and family; cloud alert is soft real-time with retries and a missed-heartbeat alarm if a device goes silent
Larger attack surface (homes, internet, cloud)	Data theft or tampering of readings	Mutual TLS, signed firmware, encryption, least-privilege access, audit and anomaly alerts
Cloud or WAN outage	Records unreachable	On-site replica with fail-over keeps local work running; devices buffer readings in flash
Failed remote firmware update could brick a device	Patient unmonitored	A/B partitions, staged roll-out, automatic rollback, watchdog
Cost and operations of two more environments	Higher administration effort	Standardise on Linux for servers, one RTOS family for monitors and home devices, centralised device management
Privacy law and provider dependence	Compliance and lock-in risk	Data-residency and consent controls (e.g. HIPAA, India's DPDP Act 2023), encrypted backups, exit plan

12.4.4 Does the plan scale to three hospitals?

Yes. Each hospital keeps its own gateway, local replica and monitors, and joins the same cloud as a separate tenant. Adding a hospital adds replicas and tenants rather than changing any OS choice, which is the strongest argument for the modified design over a rebuild.

12.4.5 Final decision

Final decision: MODIFY. Keep the hard real-time monitors, the SMP imaging server and the desktop and handheld operating systems. Add a hard real-time embedded OS to each home device with a **local** alarm under 10 ms, make the cloud a replicated distributed record with the on-site cluster as a fail-over replica, and replace trust-by-location with authentication, encryption and tenant isolation. This meets the 10 ms alarm, never-stop database and battery goals from the original scenario, and extends them safely to the home and the cloud at the lowest risk.

Task 12 summary of the rubric

Rubric part	Marks	Where addressed
Impact Analysis	2	12.1
Evaluation of the Original Design	2	12.2
Adaptation or Redesign	2	12.3
Evidence-Based Defence	2	12.4

Appendix A - README.md contents for the GitHub repository

Name: Vimala. Register number: 1924260099. Course: CSA0404 Operating Systems. Compiler: gcc 13.3.0. Linux: Ubuntu 24.04.5 LTS (kernel 6.18). Compile command: `gcc -std=c11 -Wall -Wextra -pthread -O1 -D_GNU_SOURCE file.c -o file`. To run everything: `./run_all.sh`. Per-task run commands and results:

Folder	Program	How to run	Result
Task01	latency.c	./latency ; ./latency fifo	Worst wake-up delay about 1 ms idle, about 8 ms loaded; real-time priority much smaller
Task03	ipc_cost.c	./ipc_cost	System call 83 ns, message round trip about 19.7 us
Task04	protect_demo.sh	./protect_demo.sh	Other user refused: Permission denied
Task05	readfile.c	strace ./readfile patient.txt	open, read 41, write 41, read 0, close
Task06	copy_file.c, bench_copy.c	./copy_file ; strace -c ./bench_copy rw in out 4096	10001, 21, 4 read() calls; 64 KB buffer fastest
Task07	shell_job.c, shell_job_variant.c, minishell.c, zombie_orphan.c, spawn_bench.c	./shell_job ; ./shell_job_variant | cat	Exit status 0 and 3; duplicate lines in variant; zombie and orphan shown; 1000 spawns in 0.68-0.78 s
Task08	protection.c, poll_vs_interrupt.c	./protection ; ./poll_vs_interrupt	CLI refused; polling 2.00 s CPU against 0.00 s
Task09	chase.c	./chase	1.3 ns to 167 ns across the hierarchy; cache look-up 11x faster
Task10	create_bench.c, live_mem.c, fork_vs_size.c	./create_bench	Processes 93.6 ms, threads 30.7 ms for 1,000
Task11, Task12	Analysis only (this report)	-	Plan and design challenge decisions

Commit advice: make at least one commit for each task, with a message such as *Task06: complete copy_file, add strace and time outputs*, and add screenshots of the terminal output to each folder. The faculty will check the history.